DoubleCheck · Interface Contracts (v1 draft)

The formats our two halves agree on, so Stage 1 (Rahul) and Stage 2 (Elenta) can be built in parallel and plug together. Based on the real outputs of the course SAST scanner and pentest tool.

Conventions used everywhere

Thing	Format	Example
Timestamps	ISO-8601, UTC, ms	`2026-11-03T14:22:00.123Z`
Repo	`owner/repo`	`rahul/library-app`
Job ID	UUID v4	`a7f3c1d2-...`
Severity	`HIGH` / `MEDIUM` / `LOW` (order: LOW < MEDIUM < HIGH)	`HIGH`
Scan type	`sast` / `pentest`	`sast`
Versioning	every message/item carries `schemaVersion: 1`	

---

1. Job messages (SQS)

Two queues, same envelope, different fields.

1a. SAST job → `doublecheck-sast-jobs`

Written by: webhook intake Lambda (Rahul). Read by: SAST scanner (Rahul).

```
{
  "schemaVersion": 1,
  "jobId": "a7f3c1d2-5b6e-4f1a-9c3d-2e8f7a6b5c4d",
  "scanType": "sast",
  "repo": "rahul/library-app",
  "pr": 12,
  "commitSha": "a1b2c3d4e5f6...",
  "branch": "add-login",
  "baseBranch": "main",
  "trigger": "pull_request.synchronize",
  "deliveryId": "72d3162e-cc78-11e3-81ab-4c9367dc0958",
  "requestedAt": "2026-11-03T14:22:00.123Z"
}
```

`deliveryId` is GitHub's `X-GitHub-Delivery` header, kept for debugging and duplicate detection. The scanner downloads the code itself (tarball of `commitSha` via the GitHub API) and reads `.security-gate.yml` from that commit.

1b. Pentest trigger (deploy job → API Gateway `POST /pentest`)

Written by: the app's deploy workflow (Elenta). Read by: pentest intake (Elenta).

```
{
  "repo": "rahul/library-app",
  "commitSha": "9e1f4c2...",
  "environment": "staging",
  "targetBaseUrl": "http://internal-lib-staging-123.us-east-1.elb.amazonaws.com"
}
```

Auth: header `X-DoubleCheck-Signature: sha256=<HMAC of body>` using a shared secret (GitHub secret on the app side, Secrets Manager on ours). Same verification code as the GitHub webhook.

1c. Pentest job → `doublecheck-pentest-jobs`

Written by: pentest intake (Elenta). Read by: RunTask Lambda → pentest worker (Elenta).

```
{
  "schemaVersion": 1,
  "jobId": "b8e4d2c1-...",
  "scanType": "pentest",
  "repo": "rahul/library-app",
  "commitSha": "9e1f4c2...",
  "environment": "staging",
  "targetBaseUrl": "http://internal-lib-staging-123.us-east-1.elb.amazonaws.com",
  "endpoints": ["/api/users", "/api/login", "/api/books"],
  "safeMode": false,
  "trigger": "deploy",
  "requestedAt": "2026-11-03T14:30:00.000Z"
}
```

`endpoints` and `safeMode` come from the repo's `.security-gate.yml` at `commitSha`. The tool tests one URL per call, so the worker loops over `endpoints`.

---

2. Repo config: `.security-gate.yml`

Read by both scanners. Missing file = defaults below.

```
version: 1
block_on: HIGH        # severity >= this → SAST check fails / pentest alerts
warn_on: MEDIUM       # severity >= this (but below block_on) → warning
pentest:
  endpoints:          # paths appended to targetBaseUrl
    - /api/users
    - /api/login
  safe_mode: false    # true = skip aggressive tests (rate limiting)
```

Anything below `warn_on` is ignored (not shown in comments, still stored).

Decision rule

Scan	Any finding ≥ block_on	Else any ≥ warn_on	Else

SAST	`block` (status: failure)	`warn` (status: success + warning)	`pass`
Pentest	`alert` (comment + SNS)	`warn` (comment only)	`clean`

---

3. Normalized finding

Both tools return different shapes. Each worker converts its raw output into this one shape before saving anything.

```
{
  "fingerprint": "3f9a1c0be2d47a81",
  "source": "sast",
  "ruleId": "HARDCODED_SECRET",
  "ruleName": "Hardcoded Secret",
  "severity": "HIGH",
  "status": "FAIL",
  "title": "Hardcoded password",
  "location": { "file": "src/auth.js", "line": 14, "column": 7 },
  "evidence": "const password = \"ad****\";",
  "recommendation": "Hardcoded secret detected. Move secrets to environment variables."
}
```

Pentest example:

```
{
  "fingerprint": "8c21e07fa9b35d64",
  "source": "pentest",
  "ruleId": "RATE_LIMITING",
  "ruleName": "Rate Limiting",
  "severity": "MEDIUM",
  "status": "FAIL",
  "title": "No rate limiting detected",
  "location": { "endpoint": "/api/login" },
  "evidence": "All 20 rapid requests succeeded without rate limiting",
  "recommendation": null
}
```

Mapping from raw tool output

Normalized	SAST raw	Pentest raw
ruleId	`id`	`results[].id`
ruleName	`name`	`results[].name`
severity	`severity`	`results[].severity` (test-level, copied to each finding)
status	always `FAIL`	`results[].status` (`FAIL` / `WARNING`)
title	`description`	`findings[].issue`

location	`file`, `line`, `column`	`{ endpoint }` from the job
evidence	`evidence` (masked)	`findings[].evidence` (masked)
recommendation	`message`	`findings[].recommendation` or `null`

Pentest rules: one normalized finding per nested `findings[]` item; tests with `PASS` produce none; tests with `ERROR` go in `testErrors` on the scan result, not in findings.

Fingerprint = first 16 hex chars of SHA-256 of:

- SAST: `sast|ruleId|file|evidence` (raw evidence, whitespace-trimmed; no line number, so shifted code keeps the same fingerprint)
- Pentest: `pentest|ruleId|endpoint|title`

Masking (before anything is saved or posted): inside `evidence`, any quoted string literal on a `HARDCODED_SECRET` or sensitive-data finding keeps its first 2 characters and the rest becomes `****`. The fingerprint is computed before masking.

---

4. Scan result (full report)

What each worker writes to S3 as JSON. Also the input to the results/feedback Lambda.

```
{
  "schemaVersion": 1,
  "jobId": "a7f3c1d2-...",
  "scanType": "sast",
  "repo": "rahul/library-app",
  "pr": 12,
  "commitSha": "a1b2c3d4e5f6...",
  "branch": "add-login",
  "environment": null,
  "startedAt": "2026-11-03T14:22:01.000Z",
  "finishedAt": "2026-11-03T14:22:19.400Z",
  "durationMs": 18400,
  "decision": "block",
  "reason": "1 HIGH (block_on: HIGH)",
  "configUsed": { "block_on": "HIGH", "warn_on": "MEDIUM" },
  "counts": { "HIGH": 1, "MEDIUM": 2, "LOW": 0 },
  "byType": { "HARDCODED_SECRET": 1, "WEAK_RANDOM": 2 },
  "findings": [ /* normalized findings */ ],
  "testErrors": [],
  "filesScanned": 42
}
```

For pentests: `pr` is `null` (filled in later via the merge map), `branch` is `null`, `environment` is `"staging"`, and `filesScanned` is replaced by `endpointsTested`.

---

5. S3 layout

5. S3 layout

Reports bucket: `doublecheck-reports-<accountId>` (private, block public access, SSE-S3 encryption)

```
reports/{owner}/{repo}/sast/{commitSha}/{jobId}.json
reports/{owner}/{repo}/sast/{commitSha}/{jobId}.html
reports/{owner}/{repo}/pentest/{environment}/{commitSha}/{jobId}.json
reports/{owner}/{repo}/pentest/{environment}/{commitSha}/{jobId}.html
```

Keyed by commit, not PR, because pentests start from a commit. (Changed from the explainer's `pr-12/` example.)

Lifecycle: move to Glacier after 90 days.

"View full report" links in PR comments point to `GET /reports/{jobId}` on our API, which returns a short-lived presigned S3 URL. The bucket itself never goes public.

Dashboard bucket: `doublecheck-dashboard-<accountId>` (private). Files served over HTTPS by the dashboard Lambda behind the HTTP API (CloudFront is blocked in Learner Lab).

---

6. DynamoDB tables

All on-demand capacity.

`doublecheck-scans` : **one item per completed scan**

Key	Value	Example
PK `repo`	owner/repo	`rahul/library-app`
SK `scanAt`	`finishedAt#jobId`	`2026-11-03T14:22:19.400Z#a7f3c1d2`
GSI **ByStream** PK `stream`	`repo#scanType#branchOrEnv`	`rahul/library-app#sast#add-login`
GSI **ByStream** SK `scanAt`	same as above	

Attributes: `schemaVersion, jobId, scanType, pr, commitSha, branch, environment, decision, reason, counts, byType, configUsed, fingerprints, newCount, fixedCount, durationMs, reportKey`

- History for a repo: Query `repo = :r AND scanAt BETWEEN :from AND :to`
- Previous scan: Query ByStream `stream = :s`, newest first, `Limit 2`
- Previous deployed commit (for multi-PR deploys): the previous item on stream `repo#pentest#staging`
- If `fingerprints` would push the item past ~350 KB, store them in S3 next to the report and set `fingerprintsKey` instead

`doublecheck-jobs` : **in-flight job tracking**

Key	Value

PK `jobId`	UUID
GSI **ByStatus** PK `status`	`PENDING` / `RUNNING` / `SUCCESS` / `FAILED` / `ERROR`
GSI **ByStatus** SK `createdAt`	ISO timestamp

Attributes: `scanType`, `repo`, `pr`, `commitSha`, `attempts`, `deliveryId`, `createdAt`, `updatedAt`, `error`, `expiresAt` (TTL, 30 days)

The sweeper queries ByStatus for `PENDING` and `RUNNING` with `createdAt < now − 10 min`.

`doublecheck-merges`: **commit → PR map**

Key	Value	Example
PK `repo`	owner/repo	`rahul/library-app`
SK `commitSha`	merge commit SHA	`9e1f4c2...`

Attributes: `pr`, `mergedAt`, `headBranch`. Written on `pull_request` closed + `merged = true`. Fallback: GitHub `GET /repos/{owner}/{repo}/commits/{sha}/pulls`.

---

7. GitHub conventions

Thing	Value
Status context (required check)	`DoubleCheck / SAST`
SAST comment marker	`<!-- doublecheck:sast -->` (find and update the same comment on re-scan)
Pentest comment marker	`<!-- doublecheck:pentest:{commitSha} -->` (one follow-up per deploy)
Error status text	`DoubleCheck could not complete the scan; push again to retry`

---

8. Who owns what

Contract	Writes	Reads
SAST job (1a)	Rahul (intake Lambda)	Rahul (scanner)
Pentest trigger + job (1b, 1c)	Elenta (deploy workflow, intake)	Elenta (worker)
Normalized finding / scan result (3, 4)	Both workers	Feedback Lambda, dashboard API

S3 reports (5)	Both workers	Report-link Lambda (Rahul)
Scans table	Results Lambda (Rahul)	Dashboard API (Rahul), pentest comment (Elenta)
Jobs table	Intake, workers, DLQ Lambda, sweeper	Sweeper
Merges table	Webhook Lambda (Rahul)	Pentest comment Lambda (Elenta)
PR comments (7)	SAST: Rahul · Pentest: Elenta	–

---

9. Open questions to agree on

1. Shared `normalize() + fingerprint() + mask()` as one small npm module in the DoubleCheck